

"""

=====

ANÁLISE DE TENSÕES EM TRELIÇAS PLANAS

Sistema de Análise e Otimização de Trelças Planas Isostáticas

=====

Implementação manual do método dos nós com resolução por Gauss-Jordan.

Sem uso de bibliotecas de álgebra linear (numpy.linalg, scipy, etc.).

Autor: gerado por Claude (Anthropic)

"""

```
import math
import time
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import matplotlib.cm as cm
import matplotlib.colors as mcolors
import numpy as np # apenas para geração de arrays/plots, NÃO para resolver sistemas
```

```
# =====
# MÓDULO 1 — RESOLUÇÃO DE SISTEMAS LINEARES (GAUSS-JORDAN)
# =====
```

```
def gauss_jordan(A, b):
```

```
    """
```

```
    Resolve o sistema linear  $A \cdot x = b$  pelo método de Gauss-Jordan.
```

```
    Implementação 100% manual — sem numpy.linalg, scipy ou similar.
```

```
    Parâmetros
```

```
    -----
```

```
    A : list[list[float]] Matriz dos coeficientes (n x n)
```

```
    b : list[float] Vetor independente (n)
```

```
    Retorna
```

```
    -----
```

```
    list[float] Solução x, ou lança ValueError se sistema singular.
```

```
    """
```

```
    n = len(b)
```

```
    # Monta matriz aumentada [A | b]
```

```
    M = [[A[i][j] for j in range(n)] + [b[i]] for i in range(n)]
```

```
    for col in range(n):
```

```
# Pivotamento parcial: encontra linha com maior valor absoluto
max_row = max(range(col, n), key=lambda r: abs(M[r][col]))
M[col], M[max_row] = M[max_row], M[col]
```

```
pivo = M[col][col]
if abs(pivo) < 1e-12:
    raise ValueError(
        f"Sistema singular ou mal-condicionado na coluna {col}. "
        "Verifique as condições de contorno (apoios). "
    )
```

```
# Normaliza linha do pivô
M[col] = [v / pivo for v in M[col]]
```

```
# Elimina em TODAS as outras linhas (Gauss-Jordan completo)
for row in range(n):
    if row != col:
        fator = M[row][col]
        M[row] = [M[row][j] - fator * M[col][j] for j in range(n + 1)]
```

```
return [M[i][n] for i in range(n)]
```

```
# =====
# MÓDULO 2 — MONTAGEM DA ESTRUTURA
# =====
```

```
def calcular_propriedades_barra(x, y, i, j):
    """
```

Calcula comprimento e cossenos diretores de uma barra ($i \rightarrow j$).

Retorna

(L, cx, cy) : comprimento, cos_x, cos_y

"""

```
dx = x[j] - x[i]
```

```
dy = y[j] - y[i]
```

```
L = math.hypot(dx, dy)
```

```
if L < 1e-12:
```

```
    raise ValueError(f"Barra com comprimento zero entre nós {i} e {j}.")
```

```
return L, dx / L, dy / L
```

```

def montar_matriz_equilibrio(N, B, x, y, barras, apoios):
    """
    Monta a matriz de equilíbrio  $[2N \times (B + \text{reações})]$ .

    Graus de liberdade das incógnitas
    -----
    Colunas 0..B-1      : forças nas barras
    Colunas B..B+nr-1   : reações de apoio (ordem: fixo  $\rightarrow$  Rx,Ry; móvel  $\rightarrow$  Ry)

    Retorna
    -----
    A_eq : matriz ( $2N \times n_{\text{inc}}$ )
    n_inc : número total de incógnitas
    info : dicionário com mapeamento das reações
    """
    # Contabiliza reações
    reacoes = [] # lista de (nó, direção) 0=X, 1=Y
    for no, tipo in enumerate(apoios):
        if tipo == 'fixo':
            reacoes.append((no, 0)) # Rx
            reacoes.append((no, 1)) # Ry
        elif tipo == 'movel':
            reacoes.append((no, 1)) # apenas Ry

    n_inc = B + len(reacoes)
    A_eq = [[0.0] * n_inc for _ in range(2 * N)]

    # Contribuição das barras
    for k, (i, j) in enumerate(barras):
        _, cx, cy = calcular_propriedades_barra(x, y, i, j)
        # Equilíbrio do nó i (a barra puxa com +cx,+cy se tração)
        A_eq[2 * i][k] += cx
        A_eq[2 * i + 1][k] += cy
        # Equilíbrio do nó j (sentido oposto)
        A_eq[2 * j][k] += -cx
        A_eq[2 * j + 1][k] += -cy

    # Contribuição das reações
    for idx, (no, direcao) in enumerate(reacoes):
        col = B + idx
        A_eq[2 * no + direcao][col] = 1.0

    return A_eq, n_inc, reacoes

```

```

# =====
# MÓDULO 3 — ANÁLISE PRINCIPAL
# =====

def analisar_trelica(N, x, y, barras, Fx, Fy, apoios, nome="Trelça"):
    """
    Pipeline completo de análise.

    Parâmetros
    -----
    N      : número de nós
    x, y    : coordenadas dos nós (listas de floats)
    barras  : lista de tuplas (i, j)
    Fx, Fy  : forças externas por nó
    apoios  : lista de strings ('livre', 'movel', 'fixo') por nó
    nome    : identificador para títulos dos gráficos

    Retorna
    -----
    dict com forças nas barras, reações e estatísticas
    """
    B = len(barras)
    print(f"\n{'='*60}")
    print(f"  ANÁLISE: {nome}")
    print(f"  Nós: {N}  Barras: {B}")
    print(f"{'='*60}")

    t0 = time.perf_counter()

    # ---- Monta sistema ----
    A_eq, n_inc, reacoes = montar_matriz_equilibrio(N, B, x, y, barras, apoios)

    # Verifica isostasia: 2N equações, n_inc incógnitas
    n_eq = 2 * N
    if n_eq != n_inc:
        raise ValueError(
            f"Estrutura hiperestática ou mecanismo: {n_eq} equações ≠ {n_inc} incógnitas.\n"
            "Verifique nós, barras e apoios."
        )

    # Vetor de cargas (membro direito)

```

```

b_eq = []
for i in range(N):
    b_eq.append(-Fx[i])
    b_eq.append(-Fy[i])

# ---- Resolve ----
solucao = gauss_jordan(A_eq, b_eq)

t1 = time.perf_counter()
tempo = (t1 - t0) * 1000 # ms

# ---- Extrai resultados ----
forcas = solucao[:B]
reacao_vals = solucao[B:]

# ---- Classificação ----
classificacao = []
for f in forcas:
    if abs(f) < 1e-6:
        classificacao.append('nula')
    elif f > 0:
        classificacao.append('tração')
    else:
        classificacao.append('compressão')

# ---- Estatísticas ----
magnitudes = [abs(f) for f in forcas]
idx_max = magnitudes.index(max(magnitudes))
forca_media = sum(magnitudes) / B
soma_reacoes = sum(reacao_vals)

print(f"\n {'Barra':>6} {'Nós':>8} {'Força (N)':>14} {'Tipo':>12}")
print(f" {'-'*6} {'-'*8} {'-'*14} {'-'*12}")
for k in range(B):
    i, j = barras[k]
    print(f" {k+1:>6} {i+1}—{j+1:>5} {forcas[k]:>14.4f} {classificacao[k]:>12}")

print(f"\n Reações de apoio:")
for idx, (no, direcao) in enumerate(reacoes):
    dir_str = "Rx" if direcao == 0 else "Ry"
    print(f"  Nó {no+1} {dir_str} = {reacao_vals[idx]:.4f} N")

print(f"\n ► Barra mais solicitada: {idx_max+1} ({max(magnitudes):.4f} N —

```

```
{classificacao[idx_max]})")
print(f" ▶ Força média: {forca_media:.4f} N")
print(f" ▶ Somatório das reações: {soma_reacoes:.4f} N")
print(f" ▶ Tempo de resolução: {tempo:.4f} ms")

return {
    "forças": forcas,
    "classificacao": classificacao,
    "reacoes": list(zip(reacoes, reacao_vals)),
    "magnitudes": magnitudes,
    "idx_max": idx_max,
    "forca_media": forca_media,
    "soma_reacoes": soma_reacoes,
    "tempo_ms": tempo,
}
```

```
# =====
# MÓDULO 4 — VISUALIZAÇÕES GRÁFICAS
# =====
```

```
def _cor_por_forca(forca, f_min, f_max):
    """Mapeia força → cor (azul=tração, vermelho=compressão, branco=nula)."""
    if abs(forca) < 1e-6:
        return (0.8, 0.8, 0.8)      # cinza para força nula
    norm = mcolors.TwoSlopeNorm(vmin=f_min, vcenter=0, vmax=f_max)
    cmap = cm.RdYlBu_r              # vermelho→amarelo→azul
    return cmap(norm(forca))
```

```
def desenhar_trelica_original(N, x, y, barras, apoios, Fx, Fy, ax, titulo="Treliza Original"):
    """Gráfico 1: estrutura com nós numerados, apoios e cargas."""
    ax.set_aspect('equal')
    ax.set_title(titulo, fontsize=12, fontweight='bold')

    # Barras
    for k, (i, j) in enumerate(barras):
        ax.plot([x[i], x[j]], [y[i], y[j]], 'k-', lw=2, zorder=2)
        mx, my = (x[i]+x[j])/2, (y[i]+y[j])/2
        ax.text(mx, my, str(k+1), color='navy', fontsize=7,
            ha='center', va='center',
            bbox=dict(fc='white', ec='none', alpha=0.6))
```

```

# Nós e apoios
for i in range(N):
    tipo = apoios[i]
    if tipo == 'fixo':
        ax.plot(x[i], y[i], 's', ms=12, color='#c0392b', zorder=5)
        ax.annotate('fixo', (x[i], y[i]), textcoords="offset points",
                    xytext=(-20, -15), fontsize=7, color='#c0392b')
    elif tipo == 'móvel':
        ax.plot(x[i], y[i], '^', ms=12, color='#e67e22', zorder=5)
        ax.annotate('móvel', (x[i], y[i]), textcoords="offset points",
                    xytext=(-22, -15), fontsize=7, color='#e67e22')
    else:
        ax.plot(x[i], y[i], 'o', ms=9, color='#2980b9', zorder=5)

    ax.text(x[i], y[i]+0.04*(max(y)-min(y)+1), str(i+1),
            ha='center', va='bottom', fontsize=8, fontweight='bold')

# Setas de cargas
escala = 0.15 * max(max(x)-min(x), max(y)-min(y), 1)
for i in range(N):
    if abs(Fx[i]) > 1e-9:
        ax.annotate("", xy=(x[i], y[i]),
                    xytext=(x[i] - math.copysign(escala, Fx[i]), y[i]),
                    arrowprops=dict(arrowstyle='->', color='green', lw=2))
        ax.text(x[i] - math.copysign(escala*0.5, Fx[i]), y[i] + escala*0.1,
                f"{Fx[i]:.0f}N", color='green', fontsize=7, ha='center')
    if abs(Fy[i]) > 1e-9:
        ax.annotate("", xy=(x[i], y[i]),
                    xytext=(x[i], y[i] - math.copysign(escala, Fy[i])),
                    arrowprops=dict(arrowstyle='->', color='green', lw=2))
        ax.text(x[i] + escala*0.1, y[i] - math.copysign(escala*0.5, Fy[i]),
                f"{Fy[i]:.0f}N", color='green', fontsize=7, ha='center')

ax.set_xlabel("x (m)")
ax.set_ylabel("y (m)")
margem = 0.2 * max(max(x)-min(x), max(y)-min(y), 1)
ax.set_xlim(min(x)-margem, max(x)+margem)
ax.set_ylim(min(y)-margem, max(y)+margem)
ax.grid(True, alpha=0.3)

```

```

def desenhar_trelica_colorida(N, x, y, barras, forcas, classificacao, ax,
                             titulo="Mapa de Calor das Forças Internas"):

```

""Gráfico 2: treliça colorida por intensidade (vermelho=compressão, azul=tração)."""

```
ax.set_aspect('equal')
ax.set_title(titulo, fontsize=12, fontweight='bold')

f_min = min(forcas) if min(forcas) < -1e-9 else -1
f_max = max(forcas) if max(forcas) > 1e-9 else 1
norm = mcolors.TwoSlopeNorm(vmin=f_min, vcenter=0, vmax=f_max)
cmap = cm.RdYlBu_r

# Barras coloridas com espessura proporcional
mags = [abs(f) for f in forcas]
mag_max = max(mags) if max(mags) > 0 else 1

for k, (i, j) in enumerate(barras):
    cor = cmap(norm(forcas[k]))
    lw = 1.5 + 5 * (mags[k] / mag_max)
    ax.plot([x[i], x[j]], [y[i], y[j]], '-', color=cor, lw=lw, zorder=2)

# Nós
for i in range(N):
    ax.plot(x[i], y[i], 'ko', ms=7, zorder=5)
    ax.text(x[i], y[i], f' {i+1}', fontsize=8, color='black')

# Colorbar
sm = cm.ScalarMappable(cmap=cmap, norm=norm)
sm.set_array([])
plt.colorbar(sm, ax=ax, label='Força (N)', fraction=0.046, pad=0.04)

legend_patches = [
    mpatches.Patch(color=cmap(1.0), label='Tração (+)'),
    mpatches.Patch(color=cmap(0.0), label='Compressão (-)'),
    mpatches.Patch(color='gray', label='Nula'),
]
ax.legend(handles=legend_patches, loc='upper right', fontsize=8)

ax.set_xlabel("x (m)")
ax.set_ylabel("y (m)")
margem = 0.2 * max(max(x)-min(x), max(y)-min(y), 1)
ax.set_xlim(min(x)-margem, max(x)+margem)
ax.set_ylim(min(y)-margem, max(y)+margem)
ax.grid(True, alpha=0.3)
```



```

def grafico_barras_forcas(barras, forcas, classificacao, ax,
                          titulo="Magnitude das Forças por Elemento"):
    """Gráfico 3: barras com magnitude das forças."""
    B = len(forcas)
    rotulos = [f"B{k+1}\n({barras[k][0]+1}-{barras[k][1]+1})" for k in range(B)]
    cores = []
    for c in classificacao:
        if c == 'tração':
            cores.append('#2980b9')
        elif c == 'compressão':
            cores.append('#c0392b')
        else:
            cores.append('#bdc3c7')

    bars = ax.bar(rotulos, [abs(f) for f in forcas], color=cores, edgecolor='black', lw=0.7)

    # Rótulos de valor
    for bar, f in zip(bars, forcas):
        ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + max([abs(v) for v in forcas])*0.01,
                f"{f:.1f}N", ha='center', va='bottom', fontsize=7)

    ax.set_title(titulo, fontsize=12, fontweight='bold')
    ax.set_xlabel("Barra (nós)")
    ax.set_ylabel("Força interna |F| (N)")
    ax.grid(axis='y', alpha=0.4)

    legend_patches = [
        mpatches.Patch(color='#2980b9', label='Tração (+)'),
        mpatches.Patch(color='#c0392b', label='Compressão (-)'),
        mpatches.Patch(color='#bdc3c7', label='Nula'),
    ]
    ax.legend(handles=legend_patches, fontsize=8)

def histograma_forcas(forcas, ax, titulo="Distribuição das Forças Internas"):
    """Gráfico 4: histograma da distribuição das forças."""
    n_bins = max(5, len(forcas) // 3)
    n, bins, patches = ax.hist(forcas, bins=n_bins, edgecolor='black', linewidth=0.7)

    # Colore bins: azul para positivo, vermelho para negativo
    for patch, left_edge in zip(patches, bins[:-1]):
        if left_edge >= 0:
            patch.set_facecolor('#2980b9')

```

```

else:
    patch.set_facecolor('#c0392b')

ax.axvline(0, color='black', lw=1.5, linestyle='--', label='F = 0')
media = sum(forcas) / len(forcas)
ax.axvline(media, color='green', lw=1.5, linestyle=':', label=f'Média = {media:.1f} N')

ax.set_title(titulo, fontsize=12, fontweight='bold')
ax.set_xlabel("Força interna (N)")
ax.set_ylabel("Frequência")
ax.legend(fontsize=8)
ax.grid(alpha=0.4)

def gerar_visualizacoes(N, x, y, barras, Fx, Fy, apoios, resultado, nome="Treliza"):
    """Gera os 4 gráficos obrigatórios em uma figura única."""
    fig, axes = plt.subplots(2, 2, figsize=(14, 10))
    fig.suptitle(f"Análise de Treliza — {nome} (N={N}, B={len(barras)})",
                fontsize=14, fontweight='bold', y=1.01)

    forcas = resultado["forcas"]
    classificacao = resultado["classificacao"]

    desenhar_treliza_original(N, x, y, barras, apoios, Fx, Fy, axes[0, 0],
                             titulo="1. Treliza Original — Geometria e Cargas")
    desenhar_treliza_colorida(N, x, y, barras, forcas, classificacao, axes[0, 1],
                             titulo="2. Mapa de Calor — Forças Internas")
    grafico_barras_forcas(barras, forcas, classificacao, axes[1, 0],
                          titulo="3. Magnitude das Forças por Barra")
    histograma_forcas(forcas, axes[1, 1],
                      titulo="4. Histograma da Distribuição das Forças")

    plt.tight_layout()
    fname = f"treliza_{nome.replace(' ', '_').lower()}_N{N}.png"
    plt.savefig(fname, dpi=150, bbox_inches='tight')
    print(f"\n Gráficos salvos em: {fname}")
    plt.show()

# =====
# MÓDULO 5 — TRELIÇAS DE TESTE PARAMETRIZADAS
# =====

```

```

def gerar_trelica_warren(N_half):
    """
    Gera uma treliça Warren simétrica com 2*N_half nós.
    N_half = 2 → 4 nós, 5 barras (treliça-teste do enunciado)
    N_half = 4 → 8 nós, 13 barras
    ...

    Retorna N, x, y, barras, Fx, Fy, apoios
    """
    N = 2 * N_half
    vao = N_half # comprimento total (m)
    h = 1.0      # altura

    x_coords = []
    y_coords = []

    # Nós inferiores
    for i in range(N_half):
        x_coords.append(i * vao / (N_half - 1) if N_half > 1 else 0.0)
        y_coords.append(0.0)

    # Nós superiores
    for i in range(N_half):
        x_coords.append(i * vao / (N_half - 1) if N_half > 1 else 0.0)
        y_coords.append(h)

    barras = []
    # Chord inferior
    for i in range(N_half - 1):
        barras.append((i, i + 1))
    # Chord superior
    for i in range(N_half - 1):
        barras.append((N_half + i, N_half + i + 1))
    # Diagonais e montantes
    for i in range(N_half):
        barras.append((i, N_half + i))      # montante vertical
    for i in range(N_half - 1):
        barras.append((i + 1, N_half + i))  # diagonal

    Fx = [0.0] * N
    Fy = [0.0] * N

    # Carga no nó central inferior

```

```

centro = N_half // 2
Fy[centro] = -1000.0 # 1000 N para baixo

apoios = ['livre'] * N
apoios[0] = 'fixo' # apoio fixo à esquerda (nó 0)
apoios[N_half - 1] = 'movel' # apoio móvel à direita

return N, x_coords, y_coords, barras, Fx, Fy, apoios

```

```

def trelica_teste_4nos():
    """
    Treliça Warren 4 nós / 5 barras — treliça-teste padrão do enunciado.

    Geometria:
        Nó 0 (0, 0) — apoio fixo
        Nó 1 (1, 0) — livre (carga Fy = -1000 N)
        Nó 2 (2, 0) — apoio móvel
        Nó 3 (1, 1) — livre (nó superior central)

    Barras:
        0–1 (inferior esq)
        1–2 (inferior dir)
        0–3 (diagonal esq)
        2–3 (diagonal dir)
        1–3 (montante central)
    """
    N = 4
    x = [0.0, 1.0, 2.0, 1.0]
    y = [0.0, 0.0, 0.0, 1.0]
    barras = [(0, 1), (1, 2), (0, 3), (2, 3), (1, 3)]
    Fx = [0.0, 0.0, 0.0, 0.0]
    Fy = [0.0, -1000.0, 0.0, 0.0]
    apoios = ['fixo', 'livre', 'movel', 'livre']
    return N, x, y, barras, Fx, Fy, apoios

```

```

def gerar_trelica_grande(N_target):
    """
    Gera treliça reticulada para testar escalabilidade (N = 8, 16, 32...).
    Usa a função Warren escalada para N_half = N_target // 2.
    """
    N_half = max(2, N_target // 2)

```

```
return gerar_trelica_warren(N_half)
```

```
# =====
```

```
# MÓDULO 6 — MAIN / MENU INTERATIVO
```

```
# =====
```

```
def imprimir_cabecalho():
```

```
    print("""
```

```
    ┌───────────────────────────────────────────────────────────────────────────────────┐
    │                                     │                                     │
    │ ANÁLISE DE TRELIÇAS PLANAS ISOSTÁTICAS                                     │
    │ Gauss-Jordan — sem bibliotecas de álgebra linear                             │
    │                                     │                                     │
    └───────────────────────────────────────────────────────────────────────────────────┘
    """)
```

```
def menu_principal():
```

```
    imprimir_cabecalho()
```

```
    print("Escolha o modo de operação:\n")
```

```
    print(" 1 — Treliza-teste padrão (4 nós, 5 barras — gabarito)")
```

```
    print(" 2 — Treliza Warren escalável (N = 4, 8, 16, 32)")
```

```
    print(" 3 — Entrada manual do usuário")
```

```
    print(" 4 — Benchmark de escalabilidade (N = 4, 8, 16, 32)")
```

```
    print(" 0 — Sair\n")
```

```
    return input("Opção: ").strip()
```

```
def entrada_manual():
```

```
    """Lê a estrutura interativamente do teclado."""
```

```
    print("\n--- ENTRADA MANUAL ---")
```

```
    N = int(input("Número de nós N: "))
```

```
    x, y = [], []
```

```
    for i in range(N):
```

```
        xi, yi = map(float, input(f"  Nó {i+1} — x y: ").split())
```

```
        x.append(xi); y.append(yi)
```

```
    B = int(input("Número de barras B: "))
```

```
    barras = []
```

```
    for k in range(B):
```

```
        i, j = map(int, input(f"  Barra {k+1} — nó_i nó_j (1-indexado): ").split())
```

```
        barras.append((i-1, j-1))
```

```

print("Cargas aplicadas por nó:")
Fx, Fy = [], []
for i in range(N):
    fi, fj = map(float, input(f" Nó {i+1} — Fx Fy (N): ").split())
    Fx.append(fi); Fy.append(fj)

```

```

print("Tipo de apoio por nó (livre / movel / fixo):")
apoios = []
for i in range(N):
    t = input(f" Nó {i+1}: ").strip().lower()
    apoios.append(t)

```

```

return N, x, y, barras, Fx, Fy, apoios

```

```

def benchmark():
    """Executa análise para N = 4, 8, 16, 32 e exibe tempos."""
    print("\n--- BENCHMARK DE ESCALABILIDADE ---\n")
    print(f" {'N':>5} {'Barras':>8} {'Tempo (ms)':>12} {'Maior força (N)':>18}")
    print(f" {'-'*5} {'-'*8} {'-'*12} {'-'*18}")

```

```

for N_alvo in [4, 8, 16, 32]:
    N, x, y, barras, Fx, Fy, apoios = gerar_trelica_grande(N_alvo)
    try:
        r = analisar_trelica(N, x, y, barras, Fx, Fy, apoios,
                             nome=f"Warren N={N}")
        print(f" {'N':>5} {'len(barras):>8} {r['tempo_ms']:>12.4f} "
              f"{'max(r['magnitudes']):>18.4f}")
    except ValueError as e:
        print(f" {'N':>5} ERRO: {e}")

```

```

def main():
    while True:
        opcao = menu_principal()

        if opcao == '0':
            print("Encerrando.")
            break

        elif opcao == '1':
            N, x, y, barras, Fx, Fy, apoios = trelica_teste_4nos()

```

```

    resultado = analisar_trelica(N, x, y, barras, Fx, Fy, apoios,
                                nome="Teste Padrão (4 nós)")
    gerar_visualizacoes(N, x, y, barras, Fx, Fy, apoios, resultado,
                        nome="Teste_Padrão")

elif opcao == '2':
    N_str = input("N (4, 8, 16, 32): ").strip()
    N_alvo = int(N_str)
    N, x, y, barras, Fx, Fy, apoios = gerar_trelica_grande(N_alvo)
    resultado = analisar_trelica(N, x, y, barras, Fx, Fy, apoios,
                                nome=f"Warren N={N}")
    gerar_visualizacoes(N, x, y, barras, Fx, Fy, apoios, resultado,
                        nome=f"Warren_N{N}")

elif opcao == '3':
    try:
        N, x, y, barras, Fx, Fy, apoios = entrada_manual()
        resultado = analisar_trelica(N, x, y, barras, Fx, Fy, apoios,
                                    nome="Manual")
        gerar_visualizacoes(N, x, y, barras, Fx, Fy, apoios, resultado,
                            nome="Manual")
    except Exception as e:
        print(f"\n ERRO: {e}")

elif opcao == '4':
    benchmark()
    print()

else:
    print("Opção inválida.\n")

if __name__ == "__main__":
    main()

```